

**Univerzitet u Nišu, Elektronski fakultet
Katedra za računarstvo**

Seminarski rad

Selekcija atributa u mašinskom učenju

Nemanja Kostić 1753

**Predmet: Prikupljanje i predobrada podataka za mašinsko učenje
Predmetni profesor: prof. dr Aleksandar S. Stanimirović**

Niš, 2026

Sadržaj

1. Uvod
2. Selekcija atributa – osnove
 - 2.1 Zašto je selekcija atributa važna
 - 2.2 Tipovi podataka i izbor metode
3. *Filter* metode
 - 3.1 Numerički ulaz, numerički izlaz
 - 3.2 Numerički ulaz, kategorički izlaz
 - 3.3 Kategorički ulaz, kategorički izlaz
4. *Wrapper* metode
 - 4.1 Rekurzivna eliminacija atributa (RFE)
 - 4.2 Sekvencijalni odabir atributa
5. *Embedded* metode
 - 5.1 Koeficijenti linearnih modela
 - 5.2 Stabla odlučivanja i slučajne šume
 - 5.3 Permutaciona važnost atributa
6. Poređenje metoda
7. Zaključak
8. Literatura

1. Uvod

U savremenim primenama mašinskog učenja, skupovi podataka postaju sve veći i kompleksniji. Dok su pre dvadeset godina eksperimentatori radili sa skupovima koji imaju nekoliko desetina atributa, danas je uobičajeno imati stotine, pa čak i hiljade ulaznih promenljivih. Genomika i bioinformatika tipično rade sa skupovima od više hiljada gena po uzorku, kompjuterski vid tretira svaki piksel slike kao poseban atribut (što za sliku od svega 28×28 piksela daje 784 atributa), a u obradi prirodnog jezika broj rečnika (vokabulara) koji se koristi kao skup atributa lako dostiže i stotine hiljada elemenata.

Rad sa ovoliko dimenzija nije samo tehnički izazov – on ima direktne posledice na kvalitet i interpretabilnost modela. Mnogi atributi u realnim skupovima podataka uopšte nisu relevantni za problem koji se rešava. Ako, na primer, predviđamo da li pacijent ima određenu bolest, demografski podaci poput poštanskog broja ili boje automobila ne bi smeli da utiču na predikciju, ali ako su uključeni u skup atributa, model ih može slučajno "naučiti" kao korisne prediktore.

Drugo, mnogi atributi su međusobno korelisani i ne nose novu informaciju. U meteorološkim podacima temperatura i "oseća se kao" temperatura su gotovo identični atributi – uključivanje oba u model dodaje redundanciju bez ikakve koristi. Treće, treniranje modela sa previše atributa drastično povećava vreme računanja i potrebe za memorijom, što može biti ograničavajući faktor u industrijskim primenama.

Sve ovo dovodi do fenomena poznatog kao *prokletstvo dimenzionalnosti* (eng. *curse of dimensionality*), koji Hughes opisuje kao pojavu da sa porastom broja dimenzija efikasnost klasifikatora opada ako se broj uzoraka ne povećava proporcionalno. U praksi, broj dostupnih uzoraka retko prati broj atributa.

Selekcija atributa (eng. *feature selection*) je proces pronalaženja minimalnog podskupa ulaznih promenljivih koji zadržava što je moguće više prediktivne moći originalnog skupa. Ovaj proces donosi trostruku korist: smanjuje složenost modela, ubrzava trening i, u brojnim slučajevima, poboljšava tačnost na novim podacima jer model više nije zbunjen šumom i redundancijom.

Selekciju atributa ne treba mešati sa ekstrakcijom atributa (eng. *feature extraction*). Ekstrakcija kreira potpuno nove attribute kombinacijom originalnih, dok selekcija isključivo bira podskup postojećih atributa bez ikakve transformacije njihovih vrednosti. To čini selektovane attribute direktno interpretabilnim – znamo tačno koji su originalni atributi odabrani.

U ovom seminarskom radu biće prikazane tri osnovne grupe metoda za selekciju atributa: *filter* metode, *wrapper* metode i *embedded* metode. Za svaku grupu biće objašnjen princip rada, prednosti i nedostaci.

2. Selekcija atributa – osnove

2.1 Zašto je selekcija atributa bitna

Pre nego što pređemo na konkretne tehnike, korisno je detaljnije razumeti efekte koje selekcija atributa ima na proces mašinskog učenja. Ovi efekti se mogu svrstati u tri kategorije: efekti na performansu modela, efekti na efikasnost i efekti na interpretabilnost.

Kada govorimo o performansi, intuicija je da više atributa uvek znači više informacija, pa bi model intuitivno trebalo da bude bolji. Međutim, u praksi ovo često nije slučaj. Irelevantni i redundantni atributi unose šum koji može dovesti do preteranog prilagođavanja (*overfitting*), model "upamti" specifičnosti trening skupa umesto da nauči zakonitosti. Kada se takav model primeni na nove podatke, performanse su značajno lošije.

Sa stanovišta efikasnosti, treniranje modela sa manjim brojem atributa je brže i manje memorijski zahtevno. Ovo je posebno važno kod *wrapper* metoda kao što je RFE, gde se model trenira višestruko, ali i kod samog finalnog modela koji će biti primenjen u produkciji. Manji model je i lakši za čuvanje, prenošenje i ažuriranje.

Interpretabilnost je možda najvažniji benefit u domenima kao što su medicina ili pravo, gde je neophodno razumeti zašto model donosi određenu odluku. Model koji koristi 5 atributa je znatno lakši za objašnjenje nego model koji koristi 500 atributa. Lekar može lako razumeti "pacijent ima visok krvni pritisak, visok holesterol i pušač je", ali ne može razumeti model koji koristi stotine laboratorijskih vrednosti.

2.2 Tipovi podataka i izbor metode

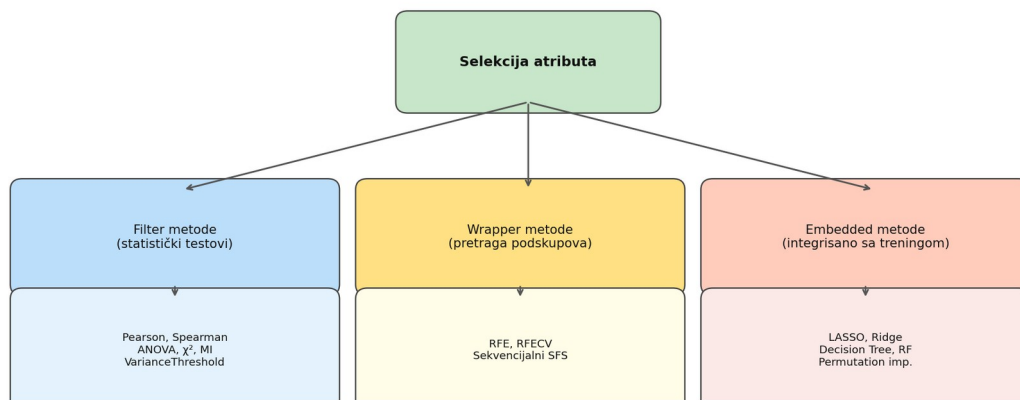
Jedan od ključnih faktora pri izboru metode selekcije atributa je tip ulaznih atributa i tip ciljne promenljive. Brownlee definiše sledeću matricu za izbor statističkih mera u *filter* metodama:

Kada su ulazni atributi numerički i ciljna promenljiva numerička (regresioni problem), koriste se mere korelacije poput Pirsonove ili Spirmanove korelacije, kao i F-statistika iz *f_regression* funkcije.

Kada su ulazni atributi numerički, a ciljna promenljiva kategorička (klasifikacioni problem), preporučuju se ANOVA F-test (*f_classif*) i uzajamna informacija (*mutual_info_classif*).

Kada su ulazni atributi kategorički i ciljna promenljiva kategorička, koriste se *chi-squared* test (*chi2*) i uzajamna informacija.

Wrapper i *embedded* metode su generalno nezavisne od tipa podataka jer se oslanjaju na sam model za ocenu važnosti atributa. Naravno, model koji se koristi mora biti adekvatan za tip problema (regresija ili klasifikacija), ali sama metoda selekcije ne postavlja ograničenja na tip podataka.



Slika 1 – Taksonomija metoda za selekciju atributa

Slika 1 – Taksonomija metoda za selekciju atributa u mašinskom učenju

Na slici 1 prikazana je taksonomija metoda za selekciju atributa. Svaka od tri grupe metoda (*filter*, *wrapper*, *embedded*) ima svoje karakteristike i primenjuje se u različitim situacijama. U nastavku rada svaka od ovih grupa biće detaljno obrađena.

3. Filter metode

Filter metode su najjednostavniji i najbrži pristup selekciji atributa. Svaki atribut se procenjuje nezavisno od ostalih, korišćenjem neke statističke mere koja kvantifikuje vezu između tog atributa i *target* promenljive. Na osnovu dobijenih rezultata, bira se top-k atributa ili svi atributi čiji skor prelazi zadati *threshold*.

Naziv "*filter*" dolazi od toga što se ove metode primenjuju kao prethodni korak pre samog treniranja modela, "filtriraju" ulazne attribute pre nego što stignu do algoritma učenja. Ovo ih razlikuje od *wrapper* i *embedded* metoda koje su jako vezane za sam algoritam.

Ključna prednost *filter* metoda je brzina i skalabilnost. Pošto ne treniraju model, mogu da obrade skupove podataka sa ogromnim brojem atributa za razumno vreme. U analizi teksta, gde mogu biti milioni atributa, *filter* metode su jedini praktični izbor. Mana je što zanemaruju interakcije između atributa: dva atributa mogu biti individualno slaba prediktora, ali zajedno nositi veliku prediktivnu moć – *filter* metode ovo neće detektovati.

Sklearn-ova *SelectKBest* klasa implementira opšti okvir za *filter* metode koji prima funkciju za računanje skorova i broj atributa *k* koji treba odabrati. Konkretna funkcija zavisi od tipa podataka, što je opisano u narednim pododeljcima.

3.1 Numerički ulaz, numerički izlaz

Kada su i ulazni atributi i ciljna promenljiva numeričke prirode (regresioni problemi), koriste se mere linearne ili monotone korelacije između atributa i ciljne promenljive.

Pirsonova korelacija je najčešće korišćena mera linearne veze između dva numerička atributa. Koeficijent *r* je u opsegu $[-1, 1]$ – vrednost $+1$ označava savršenu pozitivnu linearnu vezu, -1 savršenu negativnu, a 0 odsustvo linearne zavisnosti. Formula je:

$$r(X, y) = \frac{\sum[(X_i - \bar{X})(y_i - \bar{y})]}{\sqrt{[\sum(X_i - \bar{X})^2 \cdot \sum(y_i - \bar{y})^2]}}$$

Pirsonova korelacija pretpostavlja normalnu distribuciju podataka i osetljiva je na outliere, jedan ekstremno par vrednosti može drastično promeniti rezultat. Takođe, detektuje samo linearne veze: atribut koji je kvadratno korelisan sa ciljnom promenljivom dobiće skor blizak nuli i biće pogrešno eliminisan.

Spirmanovo rangiranje (*Spearman rank correlation*) je neparametarska alternativa Pirsonovoj korelaciji. Umesto samih vrednosti, koristi rangove. Formula je identična Pirsonovoj, ali primenjena na rangove umesto na originalne vrednosti. Ova mera detektuje monotone veze (ne samo linearne) i robusna je na outliere, pa je pogodnija za podatke koji ne zadovoljavaju pretpostavku normalnosti.

Prag varijanse (*VarianceThreshold*) je posebna *filter* metoda koja ne koristi ciljnu promenljivu – procenjuje attribute isključivo na osnovu njihove varijanse. Atributi čija je varijansa ispod zadatog praga nemaju diskriminativnu moć i eliminišu se. Na primer, atribut koji uvek ima vrednost 0 (varijansa = 0) ili atribut koji je gotovo uvek ista vrednost ne može pomoći modelu da razlikuje različite uzorke. Ovo je posebno korisno kao prvi korak pre primene zahtevnijih metoda.

```
from sklearn.feature_selection import VarianceThreshold, SelectKBest,
f_regression
from sklearn.datasets import make_regression
import numpy as np

X, y = make_regression(n_samples=500, n_features=20, n_informative=10,
random_state=1)

# Korak 1: Uklanjanje atributa sa malom varijansom
selektor_var = VarianceThreshold(threshold=0.5)
X_var = selektor_var.fit_transform(X)
print(f'Posle VarianceThreshold: {X.shape[1]} -> {X_var.shape[1]} atributa')

# Korak 2: F-statistika za regresiju (f_regression)
selektor_f = SelectKBest(score_func=f_regression, k=10)
X_novo = selektor_f.fit_transform(X, y)

# Prikaz F-skorova
skorovi = selektor_f.scores_
p_vrednosti = selektor_f.pvalues_
indeksi = np.argsort(skorovi)[::-1]
print('\nTop 5 atributa po F-skoru:')
for i in indeksi[:5]:
    print(f'  Atribut {i:2d}: F={skorovi[i]:.2f}, p={p_vrednosti[i]:.6f}')
```

3.2 Numerički ulaz, kategorički izlaz

Kada je izlaz kategorički (klasifikacioni problem), a ulazi su numerički, koristimo statističke testove koji mere razliku u distribucijama vrednosti atributa između klasa.

ANOVA F-test (*Analysis of Variance*) ispituje hipotezu da je srednja vrednost atributa ista za sve klase. Veća F-statistika znači da postoji veća razlika između klasa u pogledu vrednosti tog atributa, što sugeriše da je atribut koristan za klasifikaciju. Matematički, F-statistika se računa kao odnos varijanse između klasa i varijanse unutar klasa:

$$F = (\text{varijansa između klasa} / k - 1) / (\text{varijansa unutar klasa} / n - k)$$

gde je k broj klasa i n ukupan broj uzoraka. ANOVA F-test pretpostavlja normalnu distribuciju atributa unutar svake klase i jednakost varijansi. U sklarn-u je implementiran kao `f_classif` funkcija.

Kendalov tau je neparametarski test korelacije između rangova atributa i klasa. Pogodan je kada pretpostavke ANOVA testa nisu zadovoljene, ali daje slabije rezultate od ANOVA-e kada su pretpostavke ispunjene. Pored toga, Kendalov tau je sporiji za računanje od `f_classif`, pa se ređe koristi u praksi za veliku selekciju atributa.

Uzajamna informacija (Mutual Information, MI) meri koliko informacije jedan atribut pruža o ciljnoj promenljivoj. Matematički, definisana je kao:

$$MI(X; Y) = \sum \sum p(x,y) \cdot \log[p(x,y) / (p(x) \cdot p(y))]$$

Prednost uzajamne informacije nad ANOVA F-testom je što može detektovati nelinearne zavisnosti. Na primer, atribut koji ima kubnu vezu sa ciljnom promenljivom dobija nizak ANOVA F-skor, ali visok MI skor. Mana je što je procena MI statistički nestabilnija, posebno za male skupove podataka.

```
from sklearn.feature_selection import SelectKBest, f_classif,
mutual_info_classif
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np

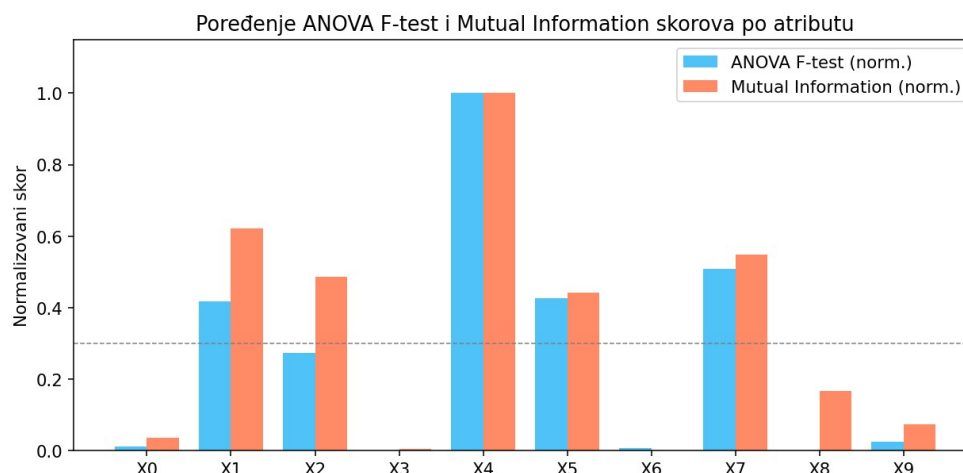
X, y = make_classification(n_samples=1000, n_features=20,
                           n_informative=10, n_redundant=5,
                           random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# ANOVA F-test - brzo i pouzdano za numeričke attribute
sel_anova = SelectKBest(score_func=f_classif, k=10)
X_tr_anova = sel_anova.fit_transform(X_train, y_train)
X_te_anova = sel_anova.transform(X_test)

# Mutual Information - detektuje nelinearne zavisnosti
sel_mi = SelectKBest(score_func=mutual_info_classif, k=10)
X_tr_mi = sel_mi.fit_transform(X_train, y_train)
X_te_mi = sel_mi.transform(X_test)

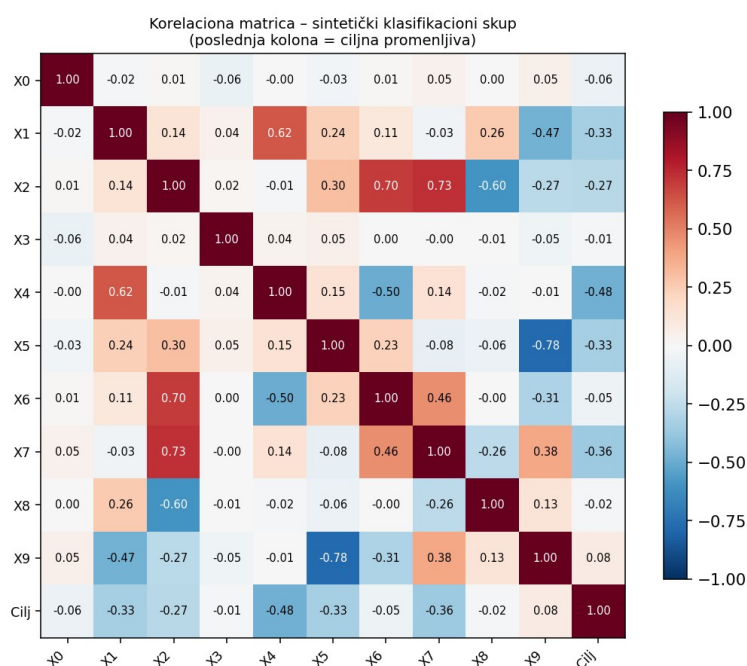
# Poređenje tačnosti
lr = LogisticRegression(max_iter=500)
for naziv, X_tr, X_te in [('ANOVA F-test', X_tr_anova, X_te_anova),
                          ('Mutual Info', X_tr_mi, X_te_mi)]:
    lr.fit(X_tr, y_train)
    tac = accuracy_score(y_test, lr.predict(X_te))
    print(f'{naziv:15s}: tačnost = {tac:.4f}')
```



Slika 3 – Normalizovani skorovi ANOVA F-test i Mutual Information metode (X0–X4 su informativni atributi)

Slika 3 – Normalizovani skorovi ANOVA F-test i Mutual Information metode za svaki atribut. Atributi X0–X4 su informativni, ostali su redundantni.

Na slici 3 prikazano je poređenje normalizovanih skorova ANOVA F-test i Mutual Information metode za sintetički skup podataka. Može se videti da obe metode uspešno identifikuju informativne attribute (X0–X4), ali im se relativni redosled razlikuje. Ovo ukazuje na to da su metode komplementarne – u praksi je korisno pogledati rezultate obe.



Slika 2 – Pirsonova korelaciona matrica između atributa i ciljne promenljive

Slika 2 – Pirsonova korelaciona matrica između atributa i ciljne promenljive. Atributi sa visokom apsolutnom korelacijom sa kolonom "Cilj" su potencijalno korisni prediktori.

3.3 Kategorički ulaz, kategorički izlaz

Kada su ulazni atributi kategoričke prirode, a i ciljna promenljiva je kategorička, koristimo drugačije mere koje su prilagođene za rad sa diskretnim podacima.

Chi-squared test (χ^2) je jedan od najstarijih i najkorišćenijih statističkih testova. Meri koliko se stvarna raspodela kategoričkih vrednosti razlikuje od očekivane raspodele ako bi atribut i ciljna promenljiva bili nezavisni. Visoka χ^2 vrednost ukazuje na jaku statističku zavisnost između atributa i ciljne promenljive, što sugerise da je atribut koristan prediktor.

Važno ograničenje *chi-squared* testa je to što vrednosti atributa moraju biti nenegativne. U *sklearn*-u, *chi2* funkcija radi direktno sa frekvencijama ili TF-IDF vrednostima, što je čini posebno pogodnom za obradu teksta.

Uzajamna informacija (*mutual_info_classif* i *mutual_info_regression*) je opšta mera koja funkcioniše i za numeričke i za kategoričke attribute. Za kategoričke attribute, MI se računa direktno iz tabele kontingencije, bez potrebe za pretpostavkama o distribuciji. Ovo je čini fleksibilnom metodom primenjivom na mešovite skupove podataka.

```
from sklearn.feature_selection import SelectKBest, chi2, mutual_info_classif
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
import numpy as np

# Simuliramo kategoričke attribute (npr. kodovane vrednosti)
np.random.seed(42)
X_cat = np.random.randint(0, 5, size=(500, 15)).astype(float)
y_cat = np.random.randint(0, 2, size=500)

# chi-squared test - vrednosti moraju biti nenegativne
sel_chi2 = SelectKBest(score_func=chi2, k=8)
X_chi2 = sel_chi2.fit_transform(X_cat, y_cat)
print(f'Chi2: odabrano {X_chi2.shape[1]} od {X_cat.shape[1]} atributa')

# Mutual Information
sel_mi = SelectKBest(score_func=mutual_info_classif, k=8)
X_mi = sel_mi.fit_transform(X_cat, y_cat)
print(f'MI: odabrano {X_mi.shape[1]} od {X_cat.shape[1]} atributa')

# Prikaz razlika u odabranim atributima
print(f'\nChi2 odabrani: {np.where(sel_chi2.get_support())[0]}')
print(f'MI odabrani: {np.where(sel_mi.get_support())[0]}')
```

4. *Wrapper* metode

Wrapper metode dobijaju ime po tome što "obmotavaju" (*wrap*) model mašinskog učenja i koriste ga kao funkciju ocenjivanja podskupova atributa. Za razliku od *filter* metoda koje procenjuju attribute statistički, *wrapper* metode direktno mere kako svaki podskup atributa utiče na performansu konkretnog modela.

Ovo čini *wrapper* metode generalno efikasnijim od *filter* metoda – one su "svesne" interakcija između atributa i toga koji je konkretan model. Međutim, cena ovakve efikasnosti je visoka računarska kompleksnost. Za *n* atributa postoji 2^n mogućih podskupova – za *n*=20 to je više od milion kombinacija, a za *n*=50 više od hiljadu milijardi. Ovaj prostor je nemoguće pretražiti u celosti, pa se koriste heuristike koje ga pametno sužavaju.

U *sklearn*-u postoje dve glavne *wrapper* metode: *Recursive Feature Elimination* (RFE) i *Sequential Feature Selector* (SFS). Obe pristupaju problemu na različit način i imaju različite prednosti.

4.1 Rekurzivna eliminacija atributa (RFE)

Rekurzivna eliminacija atributa (*Recursive Feature Elimination*, RFE) pristupa problemu "odozgo prema dole" – počinje od svih atributa i postepeno eliminiše one koji su ocenjeni kao najmanje važni. Algoritam funkcioniše u petlji:

U prvoj iteraciji trenira se model na svim atributima. Zatim se procenjuje važnost svakog atributa – za linearne modele ovo je apsolutna vrednost koeficijenta (*coef_*), za stabla je to *feature_importances_*. Atribut sa najmanjom ocenom se uklanja (ili *n_features_to_select / step* atributa). Proces se ponavlja na preostalim atributima dok se ne dostigne željeni broj.

RFE ima jedan hiperparametar koji može biti teško odrediti: koliko atributa treba odabrati (*n_features_to_select*). Ako se ovaj broj odredi ručno, može se desiti da odaberemo premalo ili previše atributa. Varijanta RFECV rešava ovaj problem automatski: za svaki mogući broj atributa (od 1 do ukupnog) procenjuje se tačnost unakrsnom validacijom, a bira se broj koji daje maksimalnu tačnost.

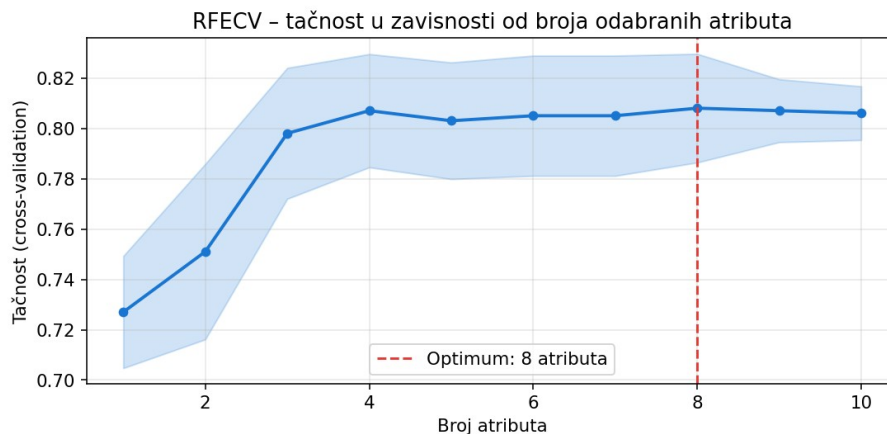
```
from sklearn.feature_selection import RFE, RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=20,
                          n_informative=10, random_state=42)

# RFE sa logističkom regresijom
lr = LogisticRegression(max_iter=1000)
rfe = RFE(estimator=lr, n_features_to_select=10, step=1)
rfe.fit(X, y)

print('Odabrani atributi:', np.where(rfe.support_)[0])
print('Rangovi atributa:')
for i, rang in enumerate(rfe.ranking_):
    status = '[ODABRAN]' if rang == 1 else f'rang {rang}'
    print(f'  Atribut {i:2d}: {status}')

# RFECV - automatski odabir optimalnog broja
rfecv = RFECV(estimator=LogisticRegression(max_iter=1000),
              cv=5, scoring='accuracy',
              min_features_to_select=1)
rfecv.fit(X, y)
print(f'\nRFECV: optimalan broj atributa = {rfecv.n_features_}')
print(f'  odabrani: {np.where(rfecv.support_)[0]}')
```



Slika 4 – RFECV kriva: tačnost unakrsne validacije u zavisnosti od broja atributa. Vertikalna linija označava automatski odabrani optimum.

Slika 4 – RFECV kriva: tačnost unakrsne validacije (5-fold) u zavisnosti od broja odabranih atributa. Crvena linija označava automatski određeni optimum (8 atributa).

Na slici 4 prikazana je RFECV kriva za sintetički skup podataka sa 10 atributa. Može se videti da tačnost raste sa brojem atributa do određene tačke (optimum je 8 atributa), a zatim se stabilizuje ili blago opada jer se dodaju redundantni atributi. RFECV automatski detektuje ovu tačku i koristi je kao rezultat selekcije.

Jedna praktična napomena: RFE je prikladniji za modele koji imaju brzu procenu važnosti atributa (linearna regresija, logistička regresija, SVM sa linearnim kernelom). Za modele kao što su neuronske mreže koji nemaju direktnu meru važnosti, RFE nije pogodan – u tom slučaju se preporučuje permutaciona važnost.

4.2 Sekvencijalni odabir atributa

Sekvencijalni odabir atributa (*Sequential Feature Selection*, SFS) je alternativni *wrapper* pristup koji ne zahteva model sa prirodnom merom važnosti atributa. Može raditi sa bilo kojim modelom koji podržava *fit()* i *score()*.

Unapred selekcija (*forward selection*) počinje od praznog skupa i gradi ga iterativno: u svakom koraku dodaje onaj atribut koji, uz već odabrane, daje najveće poboljšanje performanse. Ovo je pohlepni algoritam (*greedy*) – ne garantuje globalno optimalan podskup, ali je efikasan u praksi.

Unazad eliminacija (*backward elimination*) počinje od svih atributa i postepeno ih uklanja: u svakom koraku eliminišu onaj atribut čije uklanjanje najmanje smanjuje performansu. Unazad eliminacija je skuplja računarski (jer počinje sa više atributa), ali generalno daje bolje rezultate od unapred selekcije jer uzima u obzir interakcije između atributa.

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import numpy as np

X, y = make_classification(n_samples=500, n_features=15,
                          n_informative=8, random_state=42)

# Forward SFS sa KNN – KNN nema coef_ pa RFE ne bi radio
```

```

knn = KNeighborsClassifier(n_neighbors=5)
sfs_f = SequentialFeatureSelector(knn, n_features_to_select=5,
                                direction='forward', cv=5)
sfs_f.fit(X, y)
print(f'Forward SFS - odabrani atributi: {np.where(sfs_f.get_support())[0]}')

# Backward SFS sa logističkom regresijom
lr = LogisticRegression(max_iter=500)
sfs_b = SequentialFeatureSelector(lr, n_features_to_select=5,
                                direction='backward', cv=5)
sfs_b.fit(X, y)
print(f'Backward SFS - odabrani atributi: {np.where(sfs_b.get_support())[0]}')

# Da li se slažu?
zajednicko = set(np.where(sfs_f.get_support())[0]) &
set(np.where(sfs_b.get_support())[0])
print(f'Zajednički atributi: {sorted(zajednicko)}')

```

Jedan od nedostataka sekvencijalnog odabira je što može biti veoma spor za veliku dimenzionalnost jer svaki korak zahteva trening i evaluaciju modela za svaki preostali atribut. Na skupu sa 100 atributa, forward selekcija do 10 atributa zahteva $100 + 99 + \dots + 91 \approx 955$ evaluacija modela, pri čemu svaka evaluacija uključuje k-fold cross-validaciju. U sklearn-u je dodana podrška za paralelizaciju (`n_jobs=-1`) što značajno ubrzava proces.

5. *Embedded* metode

Embedded metode su posebna kategorija gde selekcija atributa nije odvojen korak pre ili posle treniranja modela, već se odvija integrisano tokom samog procesa treniranja. Ovo ih čini efikasnijim od *wrapper* metoda jer ne zahtevaju višestruko treniranje, ali za razliku od *filter* metoda uzimaju u obzir konkretni model i moguće interakcije između atributa.

Postoje dve glavne klase *embedded* metoda: one zasnovane na regularizaciji (LASSO, Ridge, ElasticNet) i one zasnovane na stablima odlučivanja. Posebno mesto zauzima permutaciona važnost koja tehnički može biti primenjena posle treniranja bilo kog modela, pa je ponekad svrstava u zasebnu kategoriju.

5.1 Koeficijenti linearnih modela

Linearni modeli predviđaju izlaz kao ponderisanu sumu ulaznih atributa: $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$. Svaki atribut dobija težinski koeficijent w_i , a apsolutna vrednost tog koeficijenta govori koliko promena vrednosti tog atributa utiče na predikciju modela. Atributi sa koeficijentima bliskim nuli imaju mali uticaj i mogu biti eliminirani.

Međutim, direktno poređenje koeficijenata smisleno je samo ako su atributi prethodno skalirani na isti opseg. Atribut koji meri temperaturu u Kelvinima (opseg 200-400) i atribut koji meri procenat vlage (0-100%) imaju potpuno različite skale, pa će i njihovi koeficijenti biti drugačiji, čak i ako su oba jednako važna. Zbog toga skaliranje (*StandardScaler* ili *MinMaxScaler*) mora prethoditi primeni linearnih modela za selekciju.

LASSO regularizacija (*Least Absolute Shrinkage and Selection Operator*) dodaje kaznu proporcionalnu sumi apsolutnih vrednosti koeficijenata u funkciju greške:

$$J(w) = \text{MSE}(w) + \alpha \cdot \sum |w_i|$$

Matematička posledica L1 kazne je da LASSO aktivno "gura" koeficijente prema nuli. Za dovoljno velik regularizacioni parametar α , koeficijenti nekih atributa postaju tačno nula, ti atributi su efektivno eliminirani iz modela. Ovo je suštinska razlika od Ridge regularizacije (L2), koja smanjuje koeficijente ali ih retko svodi na tačnu nulu.

ElasticNet kombinuje L1 i L2 regularizaciju: $J(w) = \text{MSE}(w) + \alpha \cdot \lambda \cdot \sum |w_i| + \alpha \cdot (1-\lambda)/2 \cdot \sum w_i^2$. Parametar λ (l1_ratio) kontroliše balans između L1 i L2. ElasticNet je posebno koristan kada postoje grupe korelisanih atributa – LASSO tipično bira jedan iz svake grupe nasumično, dok ElasticNet može da ih zadrži više.

Izbor regularizacionog parametra α je kritičan. Premali α znači slabu regularizaciju i malo eliminirani atributa. Prevelik α može eliminisati čak i informativne attribute. U praksi se α bira unakrsnom validacijom, što LassoCV i ElasticNetCV klase rade automatski.

```
from sklearn.linear_model import LassoCV, ElasticNetCV
from sklearn.feature_selection import SelectFromModel
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import numpy as np

X, y = make_regression(n_samples=500, n_features=20,
                      n_informative=8, noise=10, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Skaliranje je neophodno pre regularizacije
skaler = StandardScaler()
X_tr_sk = skaler.fit_transform(X_train)
X_te_sk = skaler.transform(X_test)

# LassoCV - unakrsna validacija za izbor alpha
lasso = LassoCV(cv=5, random_state=42, max_iter=5000)
lasso.fit(X_tr_sk, y_train)

print(f'Odabrana alpha vrednost: {lasso.alpha_:.4f}')
aktivni = np.where(np.abs(lasso.coef_) > 1e-6)[0]
print(f'Broj aktivnih atributa (LASSO): {len(aktivni)} od {X.shape[1]}')
print(f'Aktivni atributi: {aktivni}')

# SelectFromModel koristi model koji je već treniran
selektor = SelectFromModel(lasso, prefit=True)
X_tr_sel = selektor.transform(X_tr_sk)
X_te_sel = selektor.transform(X_te_sk)
print(f'\nPosle SelectFromModel: {X_tr_sel.shape[1]} atributa')

# ElasticNet kao alternativa
enet = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.9, 1.0], cv=5, random_state=42)
enet.fit(X_tr_sk, y_train)
aktivni_en = np.where(np.abs(enet.coef_) > 1e-6)[0]
print(f'Aktivni atributi (ElasticNet, l1_ratio={enet.l1_ratio_:.1f}):
{len(aktivni_en)}')
```

5.2 Stabla odlučivanja i *Random Forest*

Algoritmi zasnovani na stablima odlučivanja nude intrinzičnu meru važnosti atributa koja je direktno dostupna nakon treniranja kroz *feature_importances_property*. Ova mera je zasnovana na tome koliko svaki atribut doprinosi smanjenju nehomogenosti (*impurity*) podataka tokom procesa grananja.

Kada algoritam CART (*Classification and Regression Trees*) gradi stablo, za svaki čvor bira atribut i vrednost praga koji maksimizuju smanjenje Gini indeksa (za klasifikaciju) ili MSE (za regresiju). Ukupna "zasluga" svakog atributa je zbir smanjenja impurity-a u svim čvorovima gde je taj atribut korišćen, ponderisan brojem uzoraka koji prolaze kroz te čvorove.

Pojedinačno stablo odlučivanja ima relativno nestabilne ocene važnosti, mala promena u podacima može dovesti do sasvim drugačijeg stabla i drugačijih ocena. *Random Forest* rešava ovaj problem agregacijom: trenira se *n_estimators* stabala, svako na nasumičnom poduzorku podataka (*bootstrap*) i nasumičnom podskupu atributa, a konačna mera važnosti je prosek mera iz svih stabala.

Gradient Boosting algoritmi (XGBoost, LightGBM, GradientBoostingClassifier) takođe nude *feature_importances_property*. Za razliku od *Random Forest* koji gradi stabla paralelno i nezavisno, *Gradient Boosting* gradi stabla sekvencijalno, pri čemu svako stablo "ispravlja" greške prethodnih. Ovo ih često čini tačnijim, ali i nešto osetljivijim na prekomerno prilagođavanje.

Jedna poznata kritika mera važnosti iz stabala je tzv. *bias* prema atributima sa više mogućih vrednosti grananja. Kontinualni atributi i kategorički atributi sa mnogo vrednosti imaju više mogućih mesta grananja, pa dobijaju neproporcionalno visoke ocene čak i ako nisu stvarno informativni. Permutaciona važnost (opisana u sledećem pododeljku) ne pati od ovog problema.

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

X, y = make_classification(n_samples=1000, n_features=10,
                          n_informative=5, n_redundant=5,
                          random_state=1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42)

# Random Forest
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

# Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train, y_train)

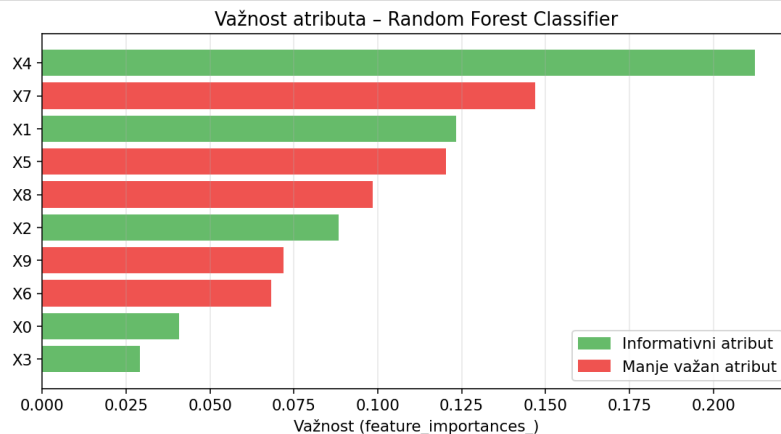
print('Važnost atributa - poređenje RF vs GB:')
print(f'{"Atribut":<10} {"RF":<10} {"GB":<10}')
print('-' * 30)
for i in range(X.shape[1]):
```

```

print(f' X{i:<8d} {rf.feature_importances_[i]:.4f}
{gb.feature_importances_[i]:.4f}')

# Selekcija 5 najvažnijih (RF)
sel = SelectFromModel(rf, max_features=5, prefit=True)
X_tr5 = sel.transform(X_train)
X_te5 = sel.transform(X_test)

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_tr5, y_train)
print(f'\nTačnost sa 5 RF-selektovanih atributa: {accuracy_score(y_test,
model.predict(X_te5)):.4f}')
```



Slika 5 – Važnost atributa prema Random Forest algoritmu. Zelenom bojom označeni su informativni, crvenom redundantni atributi.

Slika 5 – Važnost atributa prema Random Forest algoritmu. Zelenom bojom označeni su informativni (X0–X4), crvenom redundantni atributi.

Slika 5 prikazuje ocene važnosti atributa iz *Random Forest* algoritma. Može se primetiti da informativni atributi (X0–X4) dobijaju konzistentno više ocene od redundantnih atributa (X5–X9). *Random Forest* uspešno pravi razliku između ove dve grupe, što potvrđuje korisnost ove metode za selekciju atributa.

5.3 Permutaciona važnost atributa

Permutaciona važnost atributa (*Permutation Feature Importance*) je elegantna model-agnostička tehnika čiji principi su opisali Breiman (2001) u originalnom radu o *Random Forest*, a potom proširili Fisher, Rudin i Dominici (2019) na širi kontekst. Može se primeniti na bilo koji trenirani model, što je čini posebno vrednom kada model nema nativnu meru važnosti.

Algoritam se odvija u tri koraka. Najpre se model trenira i evaluira na test skupu, dobijajući baznu performansu. Zatim se, za svaki atribut j , vrednosti kolone X_j nasumično permutuju (mešaju između uzoraka) i model se ponovo evaluira na ovako izmenjenim podacima. Pad performanse u odnosu na baznu vrednost je mera važnosti atributa j . Ovaj proces se ponavlja $n_repeats$ puta kako bi se dobila statistički stabilnija ocena.

Ključna prednost ove metode je što ne pati od pristranosti prema atributima sa mnogo vrednosti grananja (za razliku od mera iz stabala), jer ne zavisi od internog mehanizma modela. Takođe, primenom na test skupu dobijamo ocenu generalizacione važnosti – koji atributi pomažu modelu na novim podacima.

Mana permutacione važnosti je što može biti *misleading* kada postoje visoko korelisani atributi. Ako su atributi X1 i X2 gotovo identični, permutacija X1 neće značajno uticati na performansu jer model može koristiti X2 kao zamenu. Oba atributa mogu dobiti niske ocene uprkos tome što nose korisnu informaciju. U takvim situacijama preporučuje se prethodno uklanjanje korelisanih atributa ili poseban tretman.

```
from sklearn.inspection import permutation_importance
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import numpy as np

X, y = make_classification(n_samples=1000, n_features=10,
                          n_informative=5, n_redundant=5,
                          random_state=1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42)

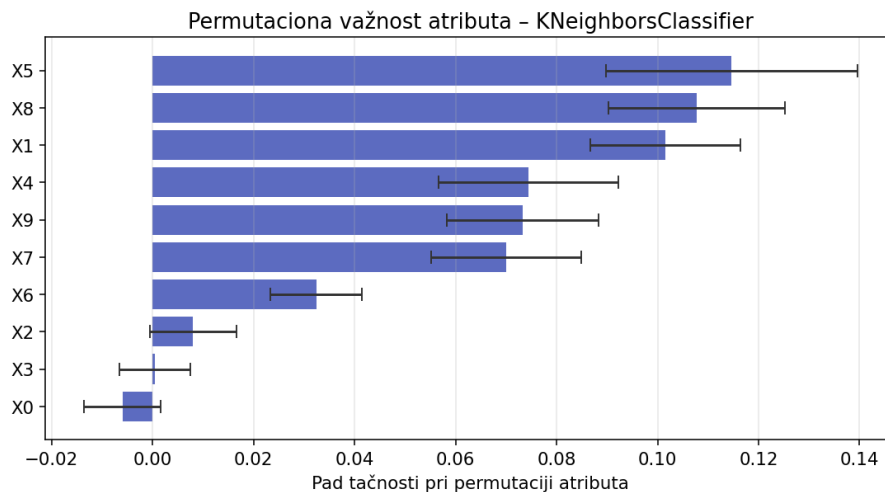
# Permutaciona važnost za KNN (koji nema nativnu meru)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# UVEK računati na TEST skupu
rez_knn = permutation_importance(knn, X_test, y_test,
                                n_repeats=15, random_state=42,
                                scoring='accuracy')

print('Permutaciona važnost (KNN, test skup):')
for i in np.argsort(rez_knn.importances_mean)[::-1]:
    print(f' X{i}: {rez_knn.importances_mean[i]:.4f} ± '
          f'{rez_knn.importances_std[i]:.4f}')

# Poređenje: RF impurity importance vs permutation importance
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rez_rf = permutation_importance(rf, X_test, y_test,
                                n_repeats=15, random_state=42,
                                scoring='accuracy')

print('\nPoređenje RF impurity vs permutation importance:')
for i in range(X.shape[1]):
    print(f' X{i}: impurity={rf.feature_importances_[i]:.4f}, '
          f'permutation={rez_rf.importances_mean[i]:.4f}')
```



Slika 6 – Permutaciona važnost atributa za KNeighborsClassifier. Horizontal bars predstavljaju srednji pad tačnosti, error bars označavaju standardnu devijaciju kroz 15 ponavljanja permutacije.

Slika 6 – Permutaciona važnost atributa za KNeighborsClassifier. Error bars prikazuju standardnu devijaciju kroz 15 ponavljanja permutacije.

Na slici 6 prikazane su permutacione važnosti svih 10 atributa. Error bars ukazuju na varijabilnost ocene kroz 15 ponavljanja – atributi sa malim error bars imaju stabilnu ocenu važnosti, dok veće error bars ukazuju na to da mešanje tog atributa ima nestabilan efekat na performansu. Informativni atributi (X0–X4) tipično imaju i veću srednju vrednost i stabilniju ocenu.

6. Poređenje metoda

Nakon što su sve tri grupe metoda opisane, korisno je sistematski sagledati njihove prednosti, mane i preporučene slučajeve primene. Ovaj pregled može poslužiti kao smernica pri izboru metode u konkretnom projektu.

Što se tiče brzine i skalabilnosti, *filter* metode su ubedljivo najbrže. Pošto ne treniraju model, mogu obraditi skupove sa milionima atributa za razumno vreme. *Embedded* metode (LASSO, RF) zahtevaju samo jedno treniranje modela i time su druge po brzini. *Wrapper* metode su najsporije jer treniraju model višestruko.

Što se tiče kvaliteta selekcije, *wrapper* metode generalno daju najbolje rezultate jer direktno optimizuju performansu konkretnog modela i uzimaju u obzir interakcije. *Embedded* metode su blizu, posebno *Random Forest* koji je robustan i stabilan. *Filter* metode mogu promašiti attribute koji su važni samo u kombinaciji sa drugima.

Interpretabilnost je slična kod svih metoda – sve daju listu odabranih atributa ili rang listu prema važnosti. Permutaciona važnost je možda najintuitivnija jer direktno odgovara na pitanje "šta se desi ako ne koristimo ovaj atribut?".

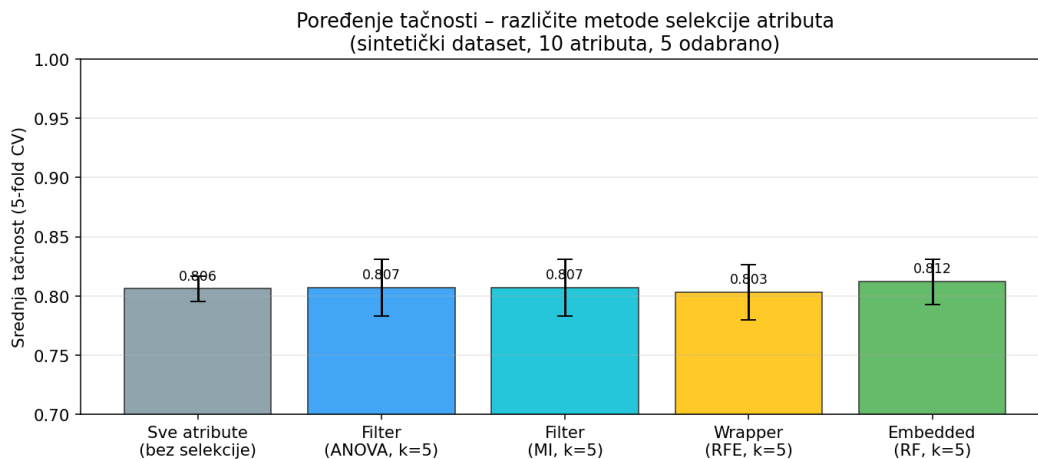
Preporučene situacije za svaku metodu:

Filter metode su idealne kada je broj atributa izuzetno velik (>1000), kada je računarsko vreme ograničeno, kada se želi brz pregled važnosti atributa pre detaljnije analize, ili kao prvi korak u višestepenom procesu selekcije. Particularly su popularne u obradi teksta (bag-of-words, TF-IDF).

Wrapper metode su preporučljive kada je broj atributa umerena (do nekoliko stotina), kada je dostupno više računarskog vremena, kada se želi optimalnost za konkretan model, ili kada su

interakcije između atributa važne. RFECV je dobar izbor kada unapred ne znamo koliko atributa treba odabrati.

Embedded metode su prirodan izbor kada se ionako koristi model koji podržava regularizaciju (linearna/logistička regresija sa LASSO) ili stabla (*Random Forest*). One kombinuju trening modela i selekciju atributa u jednom koraku, što ih čini efikasnim i jednostavnim za upotrebu u praksi.



Slika 7 – Poređenje srednje tačnosti 5-fold unakrsne validacije za različite metode selekcije.

Slika 7 – Poređenje tačnosti 5-fold unakrsne validacije za različite metode selekcije atributa na sintetičkom skupu podataka (10 atributa, biramo 5).

Na slici 7 prikazano je eksperimentalno poređenje metoda selekcije atributa na sintetičkom skupu sa 10 atributa od kojih je 5 informativno. Sve metode selekcije koje biraju 5 atributa postižu slične rezultate, a svi oni su bolji od modela koji koristi svih 10 atributa. Ovo potvrđuje teorijsku pretpostavku da eliminacija redundantnih atributa može poboljšati generalizaciju.

Treba napomenuti da će na realnim skupovima podataka razlike između metoda biti izraženije, posebno kada postoje jake interakcije između atributa ili kada su neke pretpostavke *filter* metoda narušene (npr. nelinearne zavisnosti).

7. Zaključak

U ovom seminarskom radu dat je pregled metoda za selekciju atributa u mašinskom učenju. Prikazane su tri osnovne grupe metoda – *filter*, *wrapper* i *embedded* – svaka sa svojim pristupom, prednostima i ograničenjima.

Filter metode, zasnovane na statističkim merama kao što su Pirsonova korelacija, ANOVA F-test, *chi-squared* test i uzajamna informacija, idealne su za brzu i skalabilnu selekciju. Leur je što zanemaruju interakcije između atributa i ne optimizuju direktno performansu modela.

Wrapper metode, poput RFE i sekvencijalne selekcije, direktno optimizuju performansu konkretnog modela i uzimaju u obzir interakcije. Cena je visoka računarska kompleksnost. RFECV je posebno praktičan jer automatski određuje optimalan broj atributa bez potrebe za ručnim podešavanjem.

Embedded metode – LASSO regularizacija i stabla odlučivanja – predstavljaju prirodni kompromis: selekcija je integrisana sa treningom, pa su efikasne kao filter metode, ali uzimaju u obzir model kao *wrapper* metode. Permutaciona važnost je posebno vredna jer je model-agnostička i može se primeniti posle treniranja bilo kog algoritma.

Eksperimentalno poređenje na sintetičkom skupu podataka (Slika 7) pokazalo je da sve metode uspešno identifikuju informativne attribute i poboljšavaju generalizaciju u poređenju sa modelom koji koristi sve attribute bez selekcije. U realnim primenama razlike između metoda mogu biti izraženije.

Kao zaključna preporuka: nema jednog "najboljeg" metoda za sve slučajeve. Preporučuje se početi od brze filter metode za inicijalni uvid, potom primeniti *embedded* metodu (*Random Forest* ili LASSO) za finiju selekciju, a *wrapper* metode (posebno RFECV) koristiti kada je dostupno više računarskog vremena i kada je maksimalna tačnost prioritet. Uvek treba koristiti Pipeline i unakrsnu validaciju kako bi se osigurala metodološka ispravnost evaluacije.

8. Literatura

- [1] Jason Brownlee, "Data Preparation for Machine Learning: Data Cleaning, Feature Selection, and Data Transforms in Python", Machine Learning Mastery, 2020. (Poglavljja 10–16)
- [2] L. Breiman, "Random Forests", Machine Learning, 45(1), 5–32, 2001.
- [3] A. Fisher, C. Rudin, F. Dominici, "All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously", JMLR, 20(177), 2019.
- [4] scikit-learn dokumentacija – Feature Selection:
https://scikit-learn.org/stable/modules/feature_selection.html
- [5] scikit-learn dokumentacija – Permutation Feature Importance:
https://scikit-learn.org/stable/modules/permutation_importance.html
- [6] M. Kuhn, K. Johnson, "Applied Predictive Modeling", Springer, 2013.
- [7] I. Guyon, A. Elisseeff, "An Introduction to Variable and Feature Selection", Journal of Machine Learning Research, 3, 1157–1182, 2003.